

Stack

A Stack is a linear data structure that follows the LIFO (Last In, First Out) or FILO (First In, Last Out) principle.

The element inserted last is always the first one to be removed.

All insertions and deletions take place at a single end called the top.

Core Operations:

$\text{push}(x) \rightarrow$ Inserts element x onto the top of the stack.

$\text{pop}() \rightarrow$ Removes and returns the top element.

$\text{peek}() / \text{top}() \rightarrow$ Returns the value of the top element without removing it.

$\text{isEmpty}() \rightarrow$ Returns true if the stack is containing no any elements.

$\text{isFull}() \rightarrow$ Returns true if the fixed array capacity has been reached.
[Array Only]

Note: All core operations run in $O(1)$ i.e. at constant time.

Array Implementation of Stack:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push(int value)
```

```
{
```

```
    if (top == MAX - 1)
```

```
    {
```

```
        printf("Stack is full.\n");
```

```
        return;
```

```
    }
```

```
    top ++;
```

```
    stack[top] = value;
```

```
    printf("Pushed: %d\n", value);
```

```
}
```

```
int pop()
```

```
{
```

```
    if (top == -1)
```

```
    {
```

```
        printf("Stack is empty.\n");
```

```
        return -1;
```

```
    }
```

```
    int value = stack[top];
```

```
    top --;
```

```
    return value;
```

```
}
```

```
int peek()
{
    if (top == -1)
    {
        printf("Stack is empty.\n");
        return -1;
    }

    return stack[top];
}
```

```
void display()
{
    if (top == -1)
    {
        printf("Stack is empty.\n");
        return;
    }

    for (int i = top; i >= 0; i--)
        printf("%d ", stack[i]);

    printf("\n");
}
```

```
int main()
{
    push(10);
    push(20);
    push(30);
    push(25);
}
```

```

printf("Popped: %d \n", pop());
printf("Top = %d \n", peek());
printf("Stack: ");
display();

return 0;
}

```

Output:

Pushed: 10

Pushed: 20

Pushed: 30

Pushed: 25

Popped: 25

Top = 30

Stack: 30 20 10

Linked Representation of Stack:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node
{
    int data;
    struct Node *next;
};
```

```
struct Node *top = NULL;
```

```
void push(int value)
```

```
{
    struct Node *new_node = malloc(sizeof
    (struct Node));
```

```
    new_node → data = value;
    new_node → next = top;
    top = new_node;
```

```
    printf("Pushed: %d\n", value);
```

```
}
```

```
int pop()
```

```
{
    if (top == NULL)
```

```
    {
        printf("Stack is empty.\n");
        return -1;
```

```
}
```

```
struct Node *temp = top;
```

```
int value = temp → datavalue;  
top = top → next;
```

```
free(temp);
```

```
return value;
```

```
}
```

```
int peek()  
{
```

```
    if (top == NULL)  
    {
```

```
        printf("Stack is empty.\n");  
        return -1;
```

```
    }
```

```
    return top → data;
```

```
}
```

```
void display()  
{
```

```
    if (top == NULL)  
    {
```

```
        printf("Stack is empty.\n");  
        return;
```

```
    }
```

```
    struct Node *current = top;
```

```
printf("Stack: ");
```

```
while (current != NULL)
```

```
{
```

```
    printf("%d → ", current->data);  
    current = current->next;
```

```
}
```

```
printf("NULL\n");
```

```
}
```

```
int main()
```

```
{
```

```
    push(10);
```

```
    push(20);
```

```
    push(30);
```

```
    push(25);
```

```
    printf("Popped: %d \n", pop());
```

```
    printf("Top: %d \n" peek());
```

```
    display();
```

```
    return 0;
```

```
}
```

Output:

Pushed: 10

Pushed: 20

Pushed: 30

Pushed: 25

Popped: 25

Top: 30

Stack: 30 → 20 → 10 → NULL

Infix to Postfix Conversion

Postfix or Reverse notation is a form of expression in which the operator is written after its operands.

Examples:

$$\textcircled{1} \text{ Infix} \rightarrow A + B$$

$$\text{Postfix} \rightarrow AB +$$

$$\textcircled{2} \text{ Infix} \rightarrow A + B * C$$

$$\text{Postfix} \rightarrow ABC * +$$

Here, multiplication is performed before addition.

$$\textcircled{3} \text{ Infix} \rightarrow (A + B) * C$$

$$\text{Postfix} \rightarrow AB + C * \quad \text{~~ABC + *~~}$$

C Program for Infix to ~~Postfix~~ Conversion

Postfix

Algorithm:

1. Initialize an empty operator stack and an empty output string.
2. Scan the infix expression from left to right.
 - i. If the character is an operand, ~~push~~ append it directly to the output string.
 - ii. If the character is an opening parenthesis '(', push it onto the stack.
 - iii. If the character is a closing parenthesis ')', pop operators from the stack to the output until an opening parenthesis '(' is popped.
 - iv. If the character is an operator,
 - (a) if stack is empty, push the current operator.
 - (b) if stack is not empty, pop operators with precedence "greater or equal" to current operator to the output string and then push the current operator.
 - v. Empty the stack, pop all the remaining operators to output.

Note: Precedence, $(^)>(*, /)>(+, -)$.

Program:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#define MAX 100
```

```
char stack[MAX];
int top = -1;
```

```
void push(char ch)
{
```

```
    if (top == MAX - 1)
    {
```

```
        printf("Stack is full.\n");
        return;
```

```
    }
```

```
    stack[++top] = ch;
```

```
}
```

```
char pop()
{
```

```
    if (top == -1)
    {
```

```
        printf("Stack is empty.\n");
        return '\0';
```

```
    }
```

```
    return stack[top--];
```

```
}
```

```

int precedence (char ch)
{
    if (ch == '+' || ch == '-')
        return 1;
    if (ch == '*' || ch == '/')
        return 2;
    if (ch == '^')
        return 3;
    return 0;
}

```

```

int main()
{
    char infix[100], postfix[100];
    int k = 0; // For postfix, the output string
    char ch;

    printf("Enter infix expression: ");
    fgets(infix, sizeof(infix), stdin);

    infix[strlen(infix, "\n")] = '\0';

    for (int i = 0; infix[i] != '\0'; i++)
    {
        ch = infix[i];

        if (isspace(ch))
            continue;
    }
}

```

```
else if (isalnum(ch))  
    postfix[k++] = ch;
```

```
else if (ch == '(')  
    push(ch);
```

```
else if (ch == ')')  
{
```

```
    while (top != -1 && stack[top] != '(')  
        postfix[k++] = pop();
```

```
    pop();  
}
```

```
else  
{
```

```
    while (top != -1 && stack[top] != '(' &&  
        precedence(stack[top]) >= precedence(ch)  
        && ch != '^')  
    {
```

```
        postfix[k++] = pop();  
    }
```

```
    push(ch);  
}
```

```
while (top != -1)  
    postfix[k++] = pop();
```

```
postfix[k] = '\0';
```

```
printf("\n Postfix Expression : %s", postfix),
```

```
return 0;
```

```
}
```

Output:

Enter infix expression : $(A+B)*C$

Postfix Expression : $AB+C*$

Infix to Prefix Conversion

Prefix or Polish notation is a form of expression in which the operator is written before its operands.

Examples:

$$\begin{aligned} \textcircled{1} \quad \text{Infix} &\rightarrow A + B \\ \text{Prefix} &\rightarrow +AB \end{aligned}$$

$$\begin{aligned} \textcircled{2} \quad \text{Infix} &\rightarrow A + B * C \\ \text{Prefix} &\rightarrow +A * BC \end{aligned}$$

Here, $*$ has higher precedence than $+$, so $B * C$ is grouped first.

$$\begin{aligned} \textcircled{3} \quad \text{Infix} &\rightarrow (A + B) * C \\ \text{Prefix} &\rightarrow * + ABC \end{aligned}$$

C Program for Infix to Prefix Conversion

Algorithm:

1. Scan the infix expression from right to left or reverse the expression.
2. Reverse the infix expression.
3. Replace every (with) and every) with (.
4. Convert the modified expression into postfix notation using a stack.
5. Reverse the obtained postfix expression.
6. The result is the required prefix expression.

Example:

1. $(A+B)*C$
↓
2. $C*)B+A($
↓
3. $C*(B+A)$
↓
4. $CBA+*$
↓
5. $*+ABC$

Program:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#define MAX 100
```

```
char stack stack[MAX];
top = -1;
```

```
void push(char op)
{
    stack[++top] = op;
}
```

```
char pop()
{
    return stack[top--];
}
```

```
int precedence(char op)
{
    if (op == '+' || op == '-')
        return 1;
    else if (op == '*' || op == '/')
        return 2;
    else if (op == '^')
        return 3;
    else
        return 0;
}
```

```
void reverse(char exp[])  
{
```

```
    int len = strlen(exp);  
    int i = 0, j = len - 1;  
    char temp;
```

```
    while (i < j)  
    {
```

```
        temp = exp[i];  
        exp[i] = exp[j];  
        exp[j] = temp;
```

```
        i++;
```

```
        j--;
```

```
    }
```

```
}
```

```
int main()  
{
```

```
    char infix[100], postfix[100]; prefix[100];  
    int k = 0;  
    char ch;
```

```
    printf("Enter infix expression: ");  
    scanf("%s", infix);
```

```
    reverse(infix);
```

```
    for (int i = 0; infix[i] != '\0'; i++)  
    {
```

```

        if (infix[i] == '(')
            infix[i] = ')';
        else if (infix[i] == ')')
            infix[i] = '(';
    }

```

```

for (int i = 0; infix[i] != '\0'; i++)
{

```

```

    ch = infix[i];

```

```

    if (isalnum(ch))

```

```

        postfix[k++] = ch;

```

```

    else if (ch == '(')

```

```

        push(ch);

```

```

    else if (ch == ')')

```

```

    {

```

```

        while (top != -1 && stack[top] != '(')

```

```

            postfix[k++] = pop();

```

```

    } pop();

```

```

    else

```

```

    {

```

```

        while (top != -1 && stack[top] != '('

```

```

            && (precedence(stack[top]) > precedence(ch)

```

```

            || (precedence(stack[top]) == precedence(ch)

```

```

            && ch == '^'))

```

```

        postfix[k++] = pop();
    }

```

```
    } push(ch);  
  }  
}
```

```
while (top != -1)  
    postfix[K++] = pop();
```

```
postfix[K] = '\0';
```

```
reverse(postfix);
```

```
printf("Prefix expression: %s", postfix);
```

```
return 0;
```

```
}
```

Output:

Enter infix expression: (A+B)*C

Prefix expression: *+ABC

Expression Evaluation

A stack is used to evaluate postfix and prefix expressions.

Prefix Expression Evaluation

In prefix evaluation, the expression is evaluated using a stack by scanning it from right to left.

Algorithm:

1. Scan the prefix expression from right to left.
2. If the symbol is an operand, push it onto the stack.
3. If the symbol is an operator,
 - i. Pop the top two operands.
 - ii. Apply the operator in the correct order.
 - iii. Push the result back onto the stack.
4. The final stack value is the result.

Example:

$$+A*BC \quad [\equiv A+B*C]$$

Program:

```
#include <stdio.h>
#include <ctype.h> // For isdigit() function
#include <string.h> // For strlen() function
```

```
int stack[100];
int top = -1;
```

```
void push(int value)
{
    stack[++top] = value;
}
```

```
int pop()
{
    return stack[top--];
}
```

```
int main()
{
    char exp[100];
    int i, a, b, result;
```

```
    printf("Enter prefix expression: ");
    scanf("%s", exp);
```

```
for (i = strlen(exp) - 1; i >= 0; i--)
```

```
{  
    if (isdigit(exp[i]))
```

```
        push(exp[i] - '0');
```

```
    // Convert string digit to integer. '5' to 5.
```

```
    else
```

```
{
```

```
    a = pop();
```

```
    b = pop();
```

```
    switch (exp[i])
```

```
{
```

```
        case '+':
```

```
            result = a + b;
```

```
            break;
```

```
        case '-':
```

```
            result = a - b;
```

```
            break;
```

```
        case '*':
```

```
            result = a * b;
```

```
            break;
```

```
        case '/':
```

```
            if (b == 0)
```

```
{
```

```
                printf("Error zero");
```

```
                return 1;
```

```
            }
```

```
            result = a / b;
```

```
            break;
```

```
        default:
```

```
            printf("Invalid operator");
```

```
            return 1;
```

```
    }
```

```
    push(result);  
  }  
}
```

```
printf("Result = %d", pop());
```

```
return 0;  
}
```

Output:

Enter prefix expression: $+ 2 * 35$

Result = 17

Postfix Expression Evaluation

In postfix evaluation, the expression is evaluated using a stack by scanning it from left to right.

Algorithm:

1. Scan the expression from left to right.
2. If the symbol is an operand, push it onto the stack.
3. If the symbol is an operator,
 - i. Pop the top two operands.
 - ii. Apply the operator.
 - iii. Push the result back onto the stack.
4. The final value left in the stack is the answer.

Example:

$$ABC*+ \quad [\equiv A + B * C]$$

Program:

Everything is same like for prefix evaluation program, just two changes,

- i. for loop from 0 to $\text{strlen}(\text{exp}) - 1$
- ii. the order of $\text{pop}()$ - first b then a.